

Coaching Institute Platform

A full-stack coaching institute platform built with a React + Vite frontend and a Node.js + Express + MongoDB backend. This project includes a public website, student authentication, quiz/test flows, enquiry handling, and an admin area for managing courses, users, quizzes, schools, teams, coupons, and question papers.

Features

Frontend

- React + Vite application
- Public marketing pages
- Student login and signup
- Course listing
- Quiz/test pages
- Admin dashboard UI
- Environment-based API configuration
- Deployable to Vercel

Backend

- Node.js + Express REST API
- MongoDB database integration
- Authentication and authorization
- Course, quiz, question, coupon, school, team, and user management
- Enquiry handling
- Dashboard endpoints
- Deployable to Render

Project Structure

```
coaching-institute/  
├── backend/  
│   ├── src/  
│   ├── package.json  
│   └── server.js  
└── frontend/  
    ├── src/  
    └── public/
```

```
├─ package.json
├─ vite.config.js
```

Tech Stack

Layer	Technology
Frontend	React, Vite, React Router, Tailwind CSS
Backend	Node.js, Express
Database	MongoDB
Deployment	Vercel (frontend), Render (backend)

Prerequisites

Install these before running the project:

- Node.js 18 or later
- npm
- MongoDB Atlas account or local MongoDB instance
- Git

Check installed versions:

```
node -v
npm -v
git --version
```

Clone the Project

```
git clone https://github.com/your-username/coaching-institute.git
cd coaching-institute
```

Replace `your-username` with your GitHub username if you are publishing this as a portfolio project.

Installation

Install dependencies separately for frontend and backend.

Backend setup

```
cd backend
npm install
```

Frontend setup

Open a new terminal:

```
cd frontend  
npm install
```

Environment Variables

Create environment files before starting the app.

Backend `.env`

Create this file:

```
backend/.env
```

Example:

```
PORT=5000  
MONGO_URI=your_mongodb_connection_string  
JWT_SECRET=your_super_secret_key  
NODE_ENV=development
```

Add any extra keys your backend uses, such as payment gateway secrets.

Frontend `.env`

Create this file:

```
frontend/.env
```

Example:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

For production, the frontend can use a Vercel environment variable instead of a local file:

```
VITE_API_BASE_URL=https://your-backend-name.onrender.com/api
```

Running the Project Locally

You need two terminals.

Start backend

```
cd backend  
npm run dev
```

If your project does not have a dev script, use:

```
node server.js
```

Start frontend

```
cd frontend  
npm run dev
```

Vite will usually start at:

```
http://localhost:5173
```

Backend will usually run at:

```
http://localhost:5000
```

Available Routes

Frontend

- /
- /about
- /courses
- /login
- /signup
- /enquire
- /quiz
- /take-test
- /admin/...

Backend base

- /
- /api/health
- /api/auth
- /api/courses
- /api/quizzes
- /api/questions
- /api/users
- /api/dashboard
- /api/question-papers
- /api/enquiries

Production Deployment

Frontend on Vercel

1. Push the project to GitHub.
2. Import the repository into Vercel.
3. Set the **Root Directory** to frontend.
4. Add this environment variable in Vercel:

```
VITE_API_BASE_URL=https://your-backend-name.onrender.com/api
```

5. Add frontend/vercel.json:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

This rewrite is required so React Router pages do not return 404 on refresh.

Backend on Render

1. Create a new Web Service in Render.
2. Select the same GitHub repository.
3. Set **Root Directory** to backend.

4. Build command:

```
npm install
```

5. Start command:

```
node server.js
```

6. Add backend environment variables in Render:

- PORT
- MONGO_URI
- JWT_SECRET
- any payment or third-party API keys

CORS Configuration

If frontend and backend are hosted on different domains, backend CORS must allow:

- local frontend URL
- deployed Vercel frontend URL

Example:

```
const allowedOrigins = [  
  "http://localhost:5173",  
  "https://your-frontend-name.vercel.app",  
];
```

Build Commands

Frontend production build

```
cd frontend  
npm run build
```

Frontend preview build

```
npm run preview
```

Common Issues

1. Login or signup fails in production

Cause: hardcoded localhost URLs in frontend pages or services.

Fix: use:

```
import { API_BASE_URL } from "../services/apiConfig";
```

and then:

```
fetch(`${API_BASE_URL}/auth/login`)
```

2. Refreshing a page gives 404 on Vercel

Cause: no SPA rewrite.

Fix: add `frontend/vercel.json` with rewrite to `index.html`.

3. CORS errors

Cause: backend does not allow frontend domain.

Fix: update backend CORS settings to allow localhost and Vercel frontend.

4. ERR_CONNECTION_REFUSED

Cause: frontend is still calling localhost in production.

Fix: use `VITE_API_BASE_URL` and `API_BASE_URL` everywhere.

5. Courses load but login fails

Cause: one service uses environment config while auth page still uses a hardcoded URL.

Fix: replace all direct localhost fetch calls.

Recommended Search Before Publishing

Before treating this as a polished portfolio project, search the frontend codebase for:

```
localhost  
127.0.0.1  
192.168.
```

Remove all hardcoded machine-specific URLs.

Portfolio Notes

If you are using this as a portfolio project, it helps to include:

- screenshots of the homepage and admin dashboard
- a short demo video or GIF
- a live frontend link
- a live backend health check link
- test credentials for admin and student login
- a short explanation of your role and what you built

Example section:

```
## Demo
- Live Site: https://your-frontend.vercel.app
- API Health: https://your-backend.onrender.com/api/health
- Admin Login: admin@example.com / password123
```

Scripts

These may vary depending on your package.json, but common scripts are:

Frontend

```
npm run dev
npm run build
npm run preview
```

Backend

```
npm run dev
node server.js
```

Security Notes

- Never commit real .env secrets to GitHub.
- Use different JWT secrets for development and production.
- Store production environment variables in Render and Vercel dashboards.
- Restrict CORS to trusted frontend domains.

Future Improvements

- Add forgot-password flow
- Add role-based dashboard views
- Add payment gateway production flow
- Add test result analytics
- Add image upload/storage service
- Add unit and integration tests
- Add Docker support

Author

Add your name and links here:

```
Created by Your Name
GitHub: https://github.com/your-username
LinkedIn: https://linkedin.com/in/your-profile
Portfolio: https://your-portfolio.com
```

License

Choose a license before publishing publicly.

Common choice:

```
MIT License
```